

Trabalho Prático

Programação Orientada aos Objetos

2024/2025

Grupo 44



Francisco Contente
A106846



Gustavo Braga
A107379



Lucas Pinto
A107288

1. Introdução

Este relatório descreve o projeto desenvolvido no âmbito da unidade curricular de Programação Orientada aos Objetos (POO), que teve como objetivo a criação de uma aplicação de gestão musical designada por *SpotifUM*.

Neste projeto, criamos um sistema que permite gerir e reproduzir conteúdos musicais, inspirado nas grandes plataformas de *streaming* de música. A aplicação desenvolvida permite a criação de utilizadores, que podem criar músicas, *playlists* e álbuns, mas consoante o plano que estes têm, certas ferramentas podem não estar disponíveis. Em particular, os utilizadores do tipo *premium*, têm acesso a várias funcionalidades extra, quando comparados aos utilizadores do tipo *free* (ocasionais). Para além disso, é possível realizar a consulta de estatísticas relacionadas com a utilização do programa.

Este projeto foi desenvolvido na linguagem de programação *Java*, tendo por base os princípios de programação orientada aos objetos.

2. Arquitetura da Aplicação

2.1. Arquitetura de Classes

A aplicação *SpotifUM* foi criada seguindo o padrão de arquitetura MVC (*Model-View-Controller*), que permite estabelecer uma boa estruturação e organização do código do projeto.

Os *Models* constituem a parte das classes do programa em que são representadas as entidades. Cada *Model* garante o encapsulamento dos dados de cada entidade, e inclui um conjunto de métodos que permite, aceder, modificar e gerir esse dados, de forma segura e controlada, assegurando a integridade dos dados.

As *Views* são constituídas por toda a parte de interação com o utilizador. A apresentação de conteúdos ao utilizador e a leitura das entradas do utilizador são asseguradas pelas classes como *UIs*, *Menus*, *UserInput* e *Interactive* do nosso projeto. Cabe-lhes também encaminhar os pedidos dos utilizadores para os *Controllers*, que processam as instruções recebidas e devolvem os resultados a apresentar.

Os *Controllers* são responsáveis por interligar os *Models* e as *Views*. Têm a responsabilidade de processar os pedidos vindos da interface, coordenando as ações necessárias a realizar, junto dos *Models*. Estes também asseguram que os dados são atualizados sempre que apropriado.

2.2. Entidades do Programa

2.2.1. *User*

A entidade *User* é utilizada para representar os utilizadores da aplicação. É uma classe abstrata, que serve de base para os diferentes tipos de utilizadores existentes. Os atributos base desta entidade são o *username*, nome, email, morada, aniversário, pontos e *password*.

O *UserFree* é uma herança de *User*, que representa os utilizadores com o plano gratuito. Estes têm um acesso muito limitado na plataforma, podendo apenas reproduzir *playlists* de modo aleatório e geradas pela aplicação, e consultar as estatísticas do programa. Por cada música reproduzida, ganham 5 pontos.

UserPremium é uma classe abstrata que representa os utilizadores *premium* da aplicação. Estes são diferentes dos *users free*, pois possuem uma lista de *Playlists* criadas por eles, e uma biblioteca de Álbuns privada.

UserPremiumBase é uma herança de *UserPremium*. A cada reprodução recebem 10 pontos.

UserPremiumTop é uma herança de *UserPremium*. Ao se registar na plataforma recebe 100 pontos, e a cada reprodução recebe 2.5% dos seus pontos totais.

2.2.2. Music

A componente *Music* é responsável por representar uma música dentro do sistema. Os atributos que esta entidade tem são um *id*, melodia, género, número de reproduções, título, intérprete/artista, editora, letra, duração, *id* do álbum a que pertence, e um *boolean*, a indicar se a música é ou não do tipo explícita. A classe *Music* não é abstrata porque representa músicas comuns, sem características especiais, como um vídeo. Isto evita subclasses desnecessárias e permite instanciar diretamente objetos úteis no sistema. Mantém um design simples, reutilizável e alinhado aos requisitos do projeto.

A interface *Multimedia* define a capacidade de uma música incluir conteúdo audiovisual, mais concretamente vídeo. Esta funcionalidade é implementada pela classe *MusicMultimedia*, que estende a classe *Music* e implementa a interface *Multimedia*. Assim, a *MusicMultimedia* representa músicas com vídeo associado, mantendo todas as propriedades de uma música normal, e acrescentando o campo *videoUrl*.

No programa, todas as músicas, incluindo as multimédia, estão sempre associadas a um álbum, sendo esta uma condição obrigatória na sua criação.

2.2.3. Playlist

A entidade *Playlist* representa uma coleção de músicas. É composta pelo *id*, título, lista de músicas, e 2 *booleans*, que indicam se a *playlist* é do tipo random e se é privada. É definida

como classe abstrata, que contém os atributos e comportamentos comuns a todos os tipos de *Playlists*. A partir desta classe, foram criados os diferentes tipos de *playlists*, que são heranças desta classe.

- *PlaylistRandom*, é a classe que representa as *playlists* que são criadas pelo programa, selecionando músicas aleatórias. Apenas os utilizadores do plano *Free* utilizam esta funcionalidade.
- *PremiumPlaylist* representa a *playlist* normal, criada manualmente, e unicamente por utilizadores *premium*.
- *GenrePlaylist* é uma *playlist* de apenas um género musical específico, e com um limite de tempo. Apenas os utilizadores *premium* podem criar *playlists* deste tipo.
- *FavouritePlaylist* é uma *playlist* gerada pelo programa, com base no histórico do utilizador, em que pode ser dado um limite de tempo máximo. Podem também ser criadas com apenas músicas do tipo *explicit*, conforme o que for pedido pelo utilizador *Premium*.

2.2.4. Album

A classe *Album* representa um conjunto de músicas, em que todas têm o mesmo artista associado. Os atributos presentes nesta entidade são o *id*, nome, artista, ano de lançamento e a lista de músicas do álbum. No nosso programa, estas são inicialmente criadas vazias, e, no momento de adição de uma música, esta será associada ao álbum indicado, desde que os artistas coincidam.

2.3. Controllers

Os *Controllers* desempenham um papel essencial no modelo escolhido, estabelecendo uma ponte entre os *Models* e as *Views*. Mais concretamente, os *Controllers* centralizam um conjunto de operações sobre as entidades, e regras associadas às mesmas. No nosso projeto, temos um controlador específico para gerir cada entidade, e também um *exception* específico a cada tipo de entidade, para sinalizar erros ou situações inválidas no nosso programa. Os controladores partilham várias funcionalidades em comum, mas no contexto de cada entidade, e regras específicas do programa. Algumas das funcionalidades mais comuns, são as seguintes:

- Gestão de dados: cada *Controller* contém um *Map*, que é responsável pelo armazenamento de todas as instâncias da respetiva entidade, durante a execução do programa. Cada instância está identificada por uma chave única.
- Operações de controlo comuns: para cada entidade, existem os métodos de criar, consultar, visualizar e remover objetos da entidade que gerem.
- Validação e Integridade dos Dados: Antes de adicionar ou modificar uma entidade, os *controllers* asseguram que um conjunto de regras do programa são respeitadas, permitindo lançar mensagens personalizadas de erros, quando estes ocorrem. Outra boa prática implementada para a integridade dos dados, é a realização da clonagem das entidades, sendo passados clones, e não a entidade original.

Desta forma, os controllers assumem um papel central na organização do programa, ao concentrarem toda a lógica associada às operações nas entidades. Esta separação entre os dados (*Models*) e a interface com o utilizador (*Views*) torna o código mais modular, legível e fácil de manter.

2.4. Views

As *Views* desempenham um papel essencial no sistema implementado, sendo responsáveis pela interação com o utilizador, apresentação de conteúdos, estruturação da navegação entre os diferentes menus da aplicação, e receção de *inputs*. Correspondem, essencialmente, à camada de interface entre o utilizador e a lógica do programa, encaminhando os dados introduzidos para os controladores responsáveis pelo processamento. Iremos apresentar os conteúdos que compõem esta parte da aplicação no ponto 3, onde poderão ser observados os *outputs* obtidos ao longo da interação com o programa, e a interligação dos vários conteúdos e as capacidades do nosso programa.

De modo a conseguirmos uma boa estruturação do código, criamos um conjunto de classes, que representam diferentes partes das funcionalidades de interface visual do nosso programa.

- Menu: classe com um vasto conjunto de menus do programa. Esta classe, foi criada especificamente, para integrar uma grande maioria dos menus a apresentar diretamente ao utilizador, como os menus com várias linhas e opções. No entanto, poucas são as entradas que são recebidas nos métodos implementados nesta classe.

- *MenuQueries*: disponibiliza um menu textual à visualização das estatísticas disponíveis no sistema, bem como o resultado das mesmas.
- *Input*: classe responsável por ler e validar entradas do utilizador a partir do terminal. Garante que os valores introduzidos não estão vazios e, no caso de inteiros, que são válidos. Simplifica e centraliza a gestão de *input* no programa.
- *UIs*: cada entidade do sistema possui uma *UI*. São compostas por métodos de interação com o utilizador, e também de impressão de conteúdos. Cada *UI* contém exclusivamente métodos que dizem respeito a uma entidade específica. Encontram-se aqui métodos como interação com o utilizador para criar entidades, pedir campos específicos, listar conteúdos relacionados às entidades, apresentar conteúdos conforme o tipo de utilizador ativo, entre outros. Esta distinção, é bastante vantajosa, pois permitiu-nos alcançar uma ótima estruturação do nosso código, sendo fácil de identificar onde um dado método está inserido.
- *Interactive*: parte interativa global do programa, responsável por gerir a comunicação com o utilizador. Esta camada organiza e coordena todas as funcionalidades interativas, recorrendo à chamada de métodos das diversas classes *UI*, *Menus*, e também dos controladores. Permite ter uma visão geral do que acontece no programa, e perceber a interligação entre os vários menus e pedidos do utilizador. Esta classe inclui métodos que cruzam as várias áreas do sistema, e também inclui os métodos para a realização do carregamento e salvaguarda do estado da aplicação.

2.5. Outras Classes

Para além das componentes principais do modelo MVC, existem classes auxiliares essenciais ao funcionamento da aplicação, que não se enquadram diretamente em nenhuma das três categorias do modelo. Estas classes servem propósitos de apoio, organização e estruturação interna, contribuindo para a clareza e robustez do sistema.

2.5.1. DTOs (*Data Transfer Objects*)

Existe uma classe DTO para cada entidade principal da aplicação. Cada classe inclui os parâmetros necessários que permitem, mais tarde, criar as entidades correspondentes. Estes objetos armazenam os dados fornecidos pelo utilizador sendo depois passados aos

controladores, que tratam da validação e eventual conversão para entidades do sistema. Não realizam qualquer tipo de operação ou decisão, apenas armazenam os dados. Estes objetos acabam por interagir um pouco com todas as partes da arquitetura MVC, pelo que não se inserem diretamente em nenhuma das suas componentes.

2.5.2. *Utils e Validation*

Na pasta *utils*, temos presentes as classes *Utils* e *Validation*, que integram funcionalidades de apoio ao programa. A classe *Utils* contém métodos auxiliares que, entre outras tarefas, garantem a continuidade da numeração dos identificadores das entidades que estão assim estruturadas, ao se reiniciar o programa, preservando a consistência dos dados. Um outro método aqui implementado, é a formatação do tempo total de duração das *playlists* e álbuns, numa string mais legível, ao invés de ser apresentado diretamente em segundos.

Já a classe *Validation*, faz a validação de dados introduzidos pelo utilizador, como datas, anos e palavras-passe, nos processos de criação de entidades. São implementadas nestas validações um conjunto de exceções específicas, conforme o tipo de erro detetado.

2.5.3. *Statistics*

Na pasta *queries*, encontra-se a classe responsável pelas estatísticas do sistema, *Statistics*. Nela são implementados os vários cálculos e consultas sobre os dados já existentes, permitindo obter os resultados das estatísticas do programa.

2.5.4. *Genre*

A enumeração *Genre* define os géneros musicais suportados pela aplicação, como *POP*, *ROCK*, *JAZZ*, entre outros. A inclusão do método estático *fromString(String s)* permite converter uma *String* para um valor válido da enumeração. Se a *String* fornecida não corresponder a nenhum género conhecido, é retornado por omissão o valor *POP*.

2.5.5. Reprodução

A classe *Reproducao* representa o registo de uma reprodução de música por um utilizador, contendo três informações essenciais: id do *user* que reproduziu a música, id da música reproduzida e data da reprodução.

Esta classe é essencial para consultar o histórico de reproduções de um único utilizador e realizar estatísticas do programa.

2.5.6. Estado

A classe *Estado* centraliza a gestão do estado da aplicação *SpotifUM*. Guarda as referências para os controladores principais, *UserController*, *MusicController*, *PlaylistController* e *AlbumController*, permitindo o acesso a todas as entidades do sistema.

Para além disso, armazena o *historicoGlobal*, uma lista de objetos *Reproducao* que regista todas as reproduções de músicas por utilizadores.

Ao chamar *adicionarReproducao(...)*, o sistema regista a reprodução e incrementa as *streams* da música correspondente. A classe permite a gravação e carregamento do estado completo da aplicação, assegurando a persistência dos dados do programa.

3. Descrição da Aplicação

Nesta secção, iremos apresentar a execução do programa, ilustrando a maioria das suas funcionalidades. Iremos descrever o *output* apresentado, e o comportamento do programa.

Após compilar o programa, podemos proceder à execução do mesmo. Será apresentado o seguinte menu:

```
Bem-vindo ao SpotifUM!

      .:!!--==--!!
      .-+++++++=-
      -+++++++=-
      -+++++++=-
      .+++++++=.
      !++++==-!!.....!!-+++++++.
      .+++
      .!-+++++
      =+++!.:!-+++++++=-!
      .-+++
      ++++++=-!!
      ..!-++++=!!
      .++++
      .+++++-
      .!!!!
      !+++++++=+++++++=+++++++=!
      .++++++
      =++++++-
      !-+=-+++++++=
      .++++++
      .!-=====!!
      -++++++
      !+++++++=!!
      .++++++
      -+++++++=-
      -+++++++=-
      .-+++++++=-
      .:!!--==--!!

-----MENU INICIAL-----

1) Iniciar sessão.
2) Registrar novo utilizador.
0) Sair.

Selecione a opção pretendida:
```

Figura 1 - Menu Inicial

Neste menu, temos 3 opções disponíveis. A opção 1, permite ao utilizador iniciar sessão com uma conta já existente, a opção 2, registar um novo utilizador e a opção 0, sair do programa. Vamos seleccionar a opção 2, e inserir os campos pedidos.

```
-----REGISTAR USER-----  
  
Plano de Subscrição:  
0. Free  
1. Premium Base  
2. Premium Top  
3. Consultar vantagens de cada Plano  
Opção:  
2  
Username: JSS  
Name: João Sousa  
Email: jss@gmail.com  
Address: Rua 25 de Abril, n20  
Birthday (dd/MM/yyyy): 05/07/2005  
Password:  
  
Utilizador criado com sucesso!
```

Figura 2 - Registo de um novo Utilizador

Criamos um novo utilizador, com o plano *Premium Top*. Como todos os campos solicitados foram preenchidos, o registo foi bem sucedido. A *password*, apesar de não ser visível qualquer *input*, este existe, mas a sua escrita é propositadamente ocultada.

Voltamos ao menu inicial, onde fazemos *Login*, e chegamos ao Menu do Utilizador.

```
-----LOGIN-----  
  
username: jss  
Password:  
Login bem-sucedido! Bem-vindo, João Sousa!  
  
===== MENU UTILIZADOR =====  
  
0 - Adicionar música  
1 - Reproduzir conteúdo  
2 - Criar playlist  
3 - Adicionar álbum à biblioteca  
4 - Salvar estado do programa  
5 - Carregar estado do programa  
6 - Criar álbum  
9 - Consultar estatísticas  
10 - Logout  
Escolha uma opção:
```

Figura 3 - *Login* e Menu do Utilizador

Após o *login* bem-sucedido, é apresentado o menu do utilizador, que mostra as várias funcionalidades gerais do programa. Neste caso, por se tratar de um utilizador *premium*, todas as opções estão disponíveis. Para utilizadores com plano *free*, o acesso a funcionalidades é mais limitado, pelo que o menu tem menos opções, e uma opção única de reproduzir uma *Random Playlist* gerada pelo programa.

De todas estas opções, vamos agora mostrar um conjunto delas, para evitar mostrar opções muito semelhantes, como a criação de algumas entidades, e manter a apresentação concisa. As opções escolhidas para serem mostradas no relatório são as opções 1, 2, 4 e 9. Para dar seguimento à apresentação do programa, a ordem seguida não corresponde à ordem das opções no menu original, por uma questão de lógica do programa e para obtermos uma melhor articulação entre as várias opções apresentadas.

- Opção 2 - Criar *Playlist*

```
----- Criar Nova Playlist -----
Digite o nome da playlist: Músicas Favoritas Programa

Escolha o tipo de playlist:
1 - Playlist por Género
2 - Playlist Normal
3 - Playlist de favoritos
4 - Playlist Aleatória
Opção:
3
A playlist será privada? (s/n): s

Qual a duração máxima da playlist? (em minutos)
Duração (minutos): 5
A Playlist será composta somente por musicas explicitas? (s/n): s
Playlist 'Músicas Favoritas Programa' criada com sucesso!
```

Figura 4 - Criar *Playlist* de Favoritos

Escolhendo a opção 2, podemos criar uma nova *Playlist*. Para os *Users Premium*, existem 4 tipos de *playlists* que podem ser criadas. Foi escolhida a opção “*Playlist* de favoritos”, que é uma *playlist* gerada pelo programa, com base nas reproduções do utilizador em questão. Vamos de seguida, reproduzir esta mesma *playlist*.

- Opção 1 - Reproduzir Conteúdo

```
Suas Playlists:
1 - Playlist 1 (3 músicas)
2 - Playlist Músicas Favoritas (2 músicas)
3 - Músicas Favoritas Programa (2 músicas)
4 - Gym Music (5 músicas)
5 - Rap Music Only (8 músicas)

Álbuns do User:
6 - ok (1 faixas)
7 - COR D'ÁGUA (2 faixas)

Playlists Públicas:
8 - Rap Music Only (8 músicas)

Álbuns Disponíveis:
9 - ok (1 faixas)
10 - COR D'ÁGUA (2 faixas)
11 - UTOPIA (0 faixas)
12 - ASTROWORLD (5 faixas)

0 - Voltar ao menu principal
Digite o número da opção a ser reproduzida: 3
Playlist escolhida.

Escolha o tipo de reprodução:
0 - Random
1 - Normal

1
```

Figura 5 - Selecionar *Playlist*

A reprodução de músicas é a funcionalidade central da aplicação, pois sem ela esta deixaria de cumprir o seu propósito principal enquanto plataforma musical. Ao escolhermos a opção 1, somos encaminhados para um Menu, com todas as *playlists* e álbuns disponíveis para o utilizador reproduzir. Este inclui as suas próprias *playlists*, os álbuns que adicionou à biblioteca pessoal, bem como todas as *playlists* e álbuns públicos existentes no sistema. A partir desta lista, o utilizador pode escolher uma opção, para ser iniciada a sua reprodução. Após escolher a *playlist*, o utilizador escolhe o tipo de reprodução pretendido: normal, pela ordem das músicas, quando estas foram inseridas na *playlist*, ou *random*, reproduzidas de ordem aleatória. Após esta escolha, é iniciada a reprodução.

```

Playlist • Músicas Favoritas Programa
2 músicas • 5 min
Comandos: Q - Parar | N - Próxima música | P - Música anterior

Música 1/2
Reproduzindo: SICKO MODE por Travis Scott
Duração : 150 s
Aviso: Conteúdo explícito!
Letra da música: its lit!!

n

Música 2/2
Reproduzindo: YOSEMITE por travis scott
Duração : 150 s
Aviso: Conteúdo explícito!
Letra da música: Ice on my neck
                flawless baguettes

q
Reprodução interrompida.

```

Figura 6 - Reprodução da *Playlist*

Realizou-se a reprodução da *playlist* selecionada, sendo apresentadas as músicas, respetiva letra e avisos de conteúdo explícito. Durante a execução, foram utilizados os comandos disponíveis, nomeadamente “N” para avançar para a próxima música, e “Q” para interromper a reprodução.

- Opção 9 - Consultar Estatísticas

```

===== ESTATÍSTICAS =====

1. Música mais reproduzida
2. Artista mais popular
3. Top utilizador por reproduções
4. Utilizador com mais pontos
5. Género mais popular
6. Número de playlists públicas
7. Utilizador com mais playlists
0. Voltar ao menu principal
Escolha uma opção (0-7): 5

===== QUERY 5: GÉNERO DE MÚSICA MAIS REPRODUZIDO =====

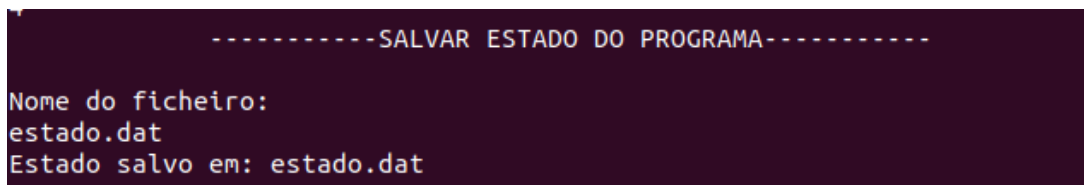
Género mais popular: RAP

```

Figura 7 - Menu de Estatísticas

Ao seleccionar a opção 9 no menu do utilizador, acedemos ao menu de estatísticas, que disponibiliza 7 tipos diferentes de estatísticas sobre os dados do sistema. Neste caso, executámos a estatística número 5, que nos indica o género mais popular do programa.

- Opção 4 - Salvar o estado do programa



```
-----SALVAR ESTADO DO PROGRAMA-----  
Nome do ficheiro:  
estado.dat  
Estado salvo em: estado.dat
```

Figura 8 - Salvar o Estado do Programa Manualmente

A opção de salvar o estado do programa permite guardar, a qualquer momento, toda a informação atual do sistema. Para isso, é solicitado o nome do ficheiro onde os dados devem ser armazenados. Esta informação é guardada num ficheiro binário, permitindo que o estado gravado possa ser recuperado mais tarde. Além disso, o estado é também guardado automaticamente ao encerrar o programa, garantindo a persistência dos dados.

Concluimos, assim, a demonstração da execução da aplicação, exemplificando algumas das principais funcionalidades disponíveis para um utilizador com plano *Premium*. Para além das opções mostradas, importa referir que o menu de utilizador com plano *Free* é mais limitado, permitindo apenas a reprodução de uma *playlist* aleatória gerada automaticamente pelo sistema, bem como o acesso às estatísticas e às funcionalidades de guardar e carregar o estado do programa. Estas restrições refletem a diferenciação funcional implementada entre os diferentes tipos de planos de subscrição, conforme os requisitos propostos.

4. Requisitos e Funcionalidades

No desenvolvimento do programa, procuramos cumprir rigorosamente os requisitos definidos no enunciado, implementando funcionalidades essenciais para a gestão e utilização do programa. A seguir, descrevemos os principais tópicos abordados:

Gestão das entidades: Permitimos a criação de utilizadores, músicas, *playlists* e álbuns, assim como o registo das reproduções efetuadas pelos utilizadores. Esta parte do programa proporciona uma base sólida para o desenvolvimento das funcionalidades seguintes da aplicação.

Estatísticas: Implementámos 7 estatísticas que permitem obter informações relevantes e frequentes, sobre o estado atual do programa.

Noção de Música Explícita: Foi incluída a funcionalidade para classificar músicas como explícitas, através de um campo booleano, permitindo que possam ser criadas *playlists* específicas com este tipo de conteúdo.

Gerar *playlists* personalizadas: Elaboramos a funcionalidade de gerar *playlists* com base no histórico de reproduções do utilizador, refletindo assim as suas preferências musicais. Esta criação pode ainda ser ajustada com restrições adicionais, como uma duração máxima total ou a inclusão exclusiva de músicas explícitas, conforme os requisitos definidos no enunciado do projeto.

Salvaguarda do estado da aplicação: A aplicação permite guardar e restaurar o estado do programa, tanto manualmente como de forma automática, no encerramento e arranque da aplicação. A informação é armazenada num ficheiro binário, garantindo a persistência dos dados e permitindo retomar a utilização sem perda de informação, o que é essencial para assegurar a continuidade e integridade da experiência do utilizador.

5. Diagrama de Classes

O diagrama de classes foi uma ferramenta essencial para o desenvolvimento do projeto, pois permitiu visualizar de forma clara as entidades do sistema, os seus atributos, métodos e as relações entre elas. Através dele, foi possível planejar melhor a estrutura do código e garantir coerência nas interações entre classes como *User*, *Music*, *Album*, *Playlist* e *Reproducao*.

As heranças estão bem definidas, como no caso dos diferentes tipos de utilizadores que estendem a classe *User*, e da classe *MusicMultimedia*, que estende de *Music*. As associações e agregações presentes (por exemplo, entre *User* e *Reproducao*, ou *Album* e *Music*) ajudaram a manter um design modular e organizado.

Com este modelo como base, a implementação tornou-se mais direta e menos propensa a erros, uma vez que a lógica do sistema já estava bem delineada.

6. Conclusão

Em suma, o nosso grupo conseguiu desenvolver com sucesso a aplicação proposta. A arquitetura do programa, baseada no modelo MVC, permitiu-nos organizar eficientemente o código, promovendo reutilização, encapsulamento e facilidade de extensão.

Este projeto foi fundamental para consolidar os conhecimentos adquiridos nas aulas, ao mesmo tempo que nos proporcionou a oportunidade de explorar novas funcionalidades e recursos da linguagem de programação *Java*.

Consideramos que o resultado final foi bastante positivo. Produzimos uma aplicação sólida, bem organizada, que cumpre os princípios da Programação Orientada aos Objetos, e está pronta para futuras evoluções. O projeto reflete o esforço e dedicação investidos por nós ao longo deste semestre.

